The output from a Django view is a response object—a web page uses one called `HttpResponse`. Django provides shortcut functions that wrap the process of loading and rendering a template and embedding the result in an `HttpResponse` object. The view returns an object to Django, and Django then uses the object to formulate the content that is sent back to the browser. This includes any HTTP headers along with the body containing the HTML.

1.1.4 Multiuser sites

From the very beginning, Django was meant for multiple users. The *Lawrence Journal-World's* web application consists of views for the readers, showing the newspaper pages as well as views for the journalists. The journalists' views are the tools they use to write articles.

Both the readers and the journalists are users, each using a different part of the web application. A regular reader should not be able to edit articles; they should only be able to read them. Controlling who can do what in a website can be broken into two parts: authentication and authorization. *Authentication* is the process of the user proving who they are, and *authorization* refers to the permissions associated with that user. Django provides tools for managing all of this.

User identities really are just another kind of data, so a multiuser Django site uses the ORM to track user accounts. Authentication compares a username and password from a login page with a known set of values in the database. Because users tend to be forgetful, you need more than just a login page—you also need a password reset screen. To manage users, you want tools for listing and editing them. You need pages for creating new users, either by an administrator or through self-sign-up. As users are people, you likely need to occasionally block one of them from logging in. Temporarily, of course—I'm sure they'll see the error of their ways. Django comes with tools to help you do all of these things. You'll learn all about users and the complications they cause in chapter 6.

In addition to providing user-management tools, Django includes a general tool called the Django Admin, which is a web-based administration interface for the database. Anything that you construct with the ORM can be managed through the Django Admin, including all of your user accounts. Having the Django Admin is a huge advantage and can save a large amount of time when building sites. The Django Admin is built on the same coding concepts as your own site: Django views, the ORM, and a series of templates. The tools that come with Django are built using the core parts of Django. You'll see how quickly you can create a site with a full administration interface in chapter 5.

1.2 What can you do with Django?

Django's original purpose at the *Lawrence Journal-World* was as a tool allowing journalists to upload articles to the web. The generic term for this kind of software is a *content management system*. Django is a general web framework, but it has a heavy leaning